

CIFAR-100 CNN Experiment: Google Colab Results Walkthrough

Deep Learning Course Assignment

This notebook trains a Convolutional Neural Network on a subset of the CIFAR-100 dataset. Several experiments were run to test how model architecture, activation function, optimizer, batch size, and class selection affected classification accuracy.

Dataset: CIFAR-100, 10 selected classes

Framework: PyTorch

Hardware: Google Colab Free GPU runtime, Tesla T4

1. Purpose of this notebook

This notebook documents the CIFAR-100 CNN experiments run in Google Colab Free using a Tesla T4 GPU runtime. The goal was to train a convolutional neural network from scratch on a smaller 10-class subset of CIFAR-100 and evaluate how different training choices affected classification accuracy.

The code is kept visible because it shows the full workflow used in the experiments, including environment checking, dataset preprocessing, tensor conversion, model definition, training loop, evaluation, and result visualization.

2. Environment

The notebook was run in Google Colab Free using PyTorch and torchvision. GPU acceleration was available.

The reported environment was:

- PyTorch version: 2.10.0+cu128
- GPU available: True
- GPU: Tesla T4

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
import torchvision
import numpy as np
import matplotlib.pyplot as plt

print(f"PyTorch version: {torch.__version__}")
print(f"GPU available: {torch.cuda.is_available()}")
```

```
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
```

Listing 1. Import statements and Google Colab GPU environment check.

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
import torchvision
import numpy as np
import matplotlib.pyplot as plt

print(f"PyTorch version: {torch.__version__}")
print(f"GPU available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")

PyTorch version: 2.10.0+cu128
GPU available: True
GPU: Tesla T4
```

Figure 1. Google Colab Free environment showing PyTorch with CUDA support and a Tesla T4 GPU.

3. Data loading and preprocessing

The CIFAR-100 dataset was used for the image classification task. Instead of using all 100 classes, the notebook selected a smaller subset of 10 classes to keep the experiment manageable in Google Colab Free.

The first selected subset used CIFAR-100 classes 0–9:

apple, aquarium_fish, baby, bear, beaver, bed, bee, beetle, bicycle, and bottle.

These classes were chosen because they are relatively visually distinct from each other, making them a reasonable baseline for a 10-class image classification task.

After filtering, the dataset contained:

- Training samples: **5,000**
- Test samples: **1,000**
- Image shape: **32 × 32 × 3**

The pixel values were normalized to the range 0–1 before being converted into PyTorch tensors.

```
# Load CIFAR-100 dataset
trainset = torchvision.datasets.CIFAR100(root='./data', train=True,
download=True)
testset = torchvision.datasets.CIFAR100(root='./data', train=False,
```

```

download=True)

X_train = np.array(trainset.data)
y_train = np.array(trainset.targets)
X_test = np.array(testset.data)
y_test = np.array(testset.targets)

# Filter to 10 classes: classes 0-9
selected_classes = np.arange(10)

class_names = [
    'apple', 'aquarium_fish', 'baby', 'bear', 'beaver',
    'bed', 'bee', 'beetle', 'bicycle', 'bottle'
]

train_mask = np.isin(y_train, selected_classes)
X_train, y_train = X_train[train_mask], y_train[train_mask]

test_mask = np.isin(y_test, selected_classes)
X_test, y_test = X_test[test_mask], y_test[test_mask]

# Normalize pixel values to range 0-1
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0

print(f"Training samples: {len(X_train)}")
print(f"Test samples: {len(X_test)}")
print(f"Image shape: {X_train[0].shape}")

```

Listing 2. CIFAR-100 loading, 10-class filtering, and normalization.

```

plt.figure(figsize=(10, 10))

for i in range(16):
    plt.subplot(4, 4, i + 1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(X_train[i])
    plt.xlabel(class_names[y_train[i]])

plt.suptitle('Sample Training Images')
plt.tight_layout()
plt.show()

```

Listing 3. Sample image plotting code for the selected CIFAR-100 subset.

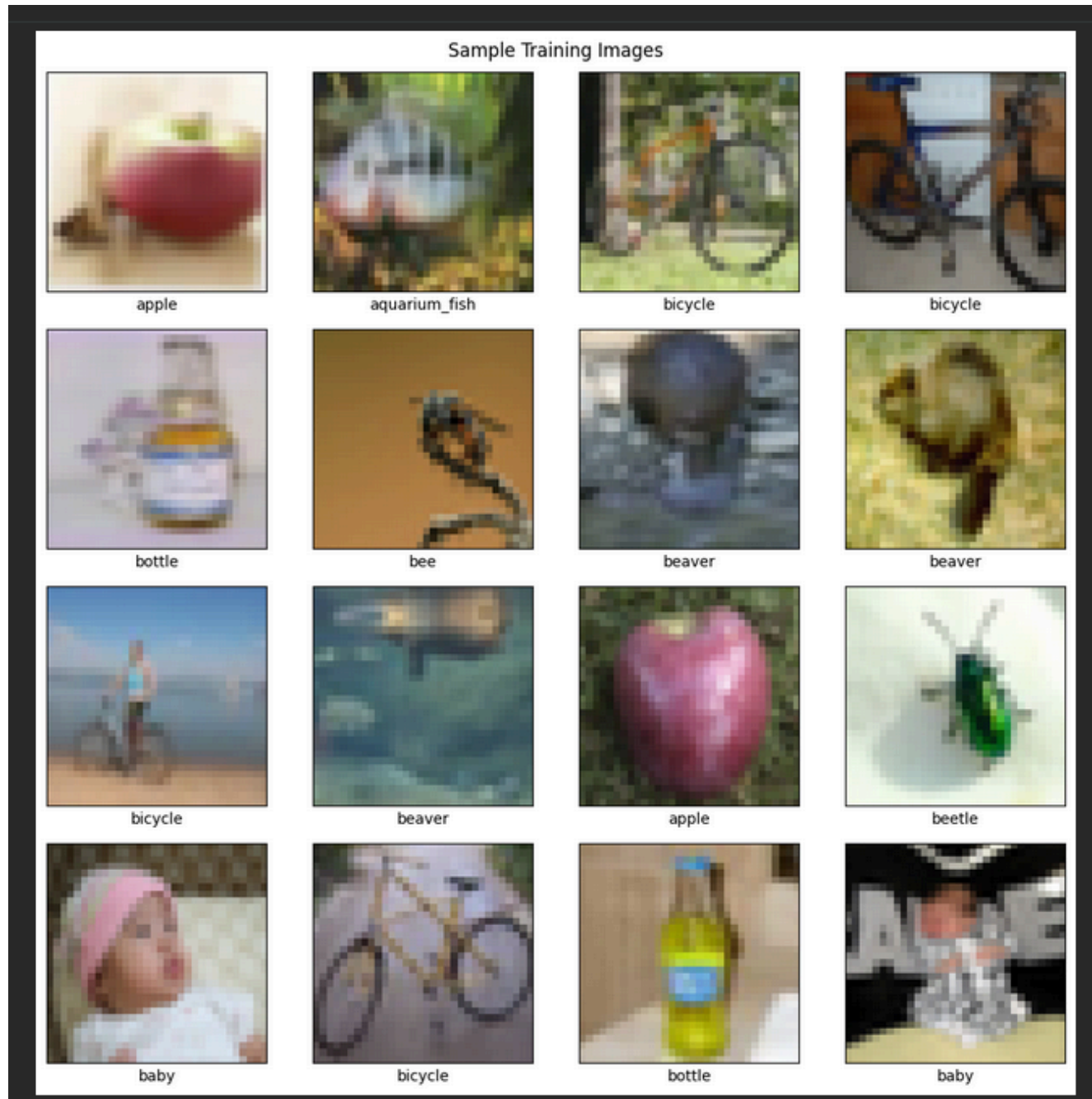


Figure 2. Example images from the selected 10-class CIFAR-100 training subset.

```
X_train_t = torch.tensor(X_train).permute(0, 3, 1, 2)
y_train_t = torch.tensor(y_train, dtype=torch.long)

X_test_t = torch.tensor(X_test).permute(0, 3, 1, 2)
y_test_t = torch.tensor(y_test, dtype=torch.long)

train_loader = DataLoader(
    TensorDataset(X_train_t, y_train_t),
    batch_size=64,
    shuffle=True
)

test_loader = DataLoader(
    TensorDataset(X_test_t, y_test_t),
```

```

        batch_size=64,
        shuffle=False
    )

    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    print(f"Training on: {device}")

```

Listing 4. Tensor conversion, DataLoader creation, and device selection.

4. Baseline model

The baseline model was a convolutional neural network trained from scratch. It used three convolutional blocks with increasing filters: 32, 64, and 128. Each block used two convolutional layers, ReLU activation, max pooling, and dropout. The classifier section flattened the learned feature maps and passed them through a dense layer before producing predictions for the 10 selected classes.

The baseline configuration was:

- Activation: ReLU
- Optimizer: Adam
- Learning rate: 0.001
- Batch size: 64
- Epochs: 30

The baseline reached **61.5%** validation accuracy according to the final summary table in the notebook. The printed plot output for this run showed **62.90%**, but the final summary table uses **61.5%** as the baseline result, so that value is used for the results summary.

```

class CNN(nn.Module):
    def __init__(self, activation=nn.ReLU):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv2d(3, 32, (3, 3), padding=1), activation(),
            nn.Conv2d(32, 32, (3, 3), padding=1), activation(),
            nn.MaxPool2d((2, 2)), nn.Dropout2d(0.25),

            nn.Conv2d(32, 64, (3, 3), padding=1), activation(),
            nn.Conv2d(64, 64, (3, 3), padding=1), activation(),
            nn.MaxPool2d((2, 2)), nn.Dropout2d(0.25),

            nn.Conv2d(64, 128, (3, 3), padding=1), activation(),
            nn.Conv2d(128, 128, (3, 3), padding=1), activation(),
            nn.MaxPool2d((2, 2)), nn.Dropout2d(0.25),
        )

```

```

self.classifier = nn.Sequential(
    nn.Flatten(),
    nn.Linear(2048, 512), activation(),
    nn.Dropout(0.5),
    nn.Linear(512, 10),
)

def forward(self, x):
    x = self.features(x)
    x = self.classifier(x)
    return x

```

Listing 5. Baseline CNN model definition.

```

def train_model(model, train_loader, test_loader, optimizer, epochs=30):
    criterion = nn.CrossEntropyLoss()
    train_accs, val_accs, train_losses, val_losses = [], [], [], []

    for epoch in range(epochs):
        model.train()
        correct, total, running_loss = 0, 0, 0.0

        for X_batch, y_batch in train_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)

            optimizer.zero_grad()
            outputs = model(X_batch)
            loss = criterion(outputs, y_batch)
            loss.backward()
            optimizer.step()

            running_loss += loss.item()
            correct += (outputs.argmax(1) == y_batch).sum().item()
            total += y_batch.size(0)

        train_accs.append(correct / total)
        train_losses.append(running_loss / len(train_loader))

        model.eval()
        correct, total, running_loss = 0, 0, 0.0

        with torch.no_grad():
            for X_batch, y_batch in test_loader:
                X_batch, y_batch = X_batch.to(device), y_batch.to(device)

                outputs = model(X_batch)
                loss = criterion(outputs, y_batch)

                running_loss += loss.item()

```

```

        correct += (outputs.argmax(1) == y_batch).sum().item()
        total += y_batch.size(0)

    val_accs.append(correct / total)
    val_losses.append(running_loss / len(test_loader))

    print(
        f"Epoch {epoch + 1}/{epochs} | "
        f"Train acc: {train_accs[-1]:.4f} | "
        f"Val acc: {val_accs[-1]:.4f}"
    )

    return train_accs, val_accs, train_losses, val_losses

```

Listing 6. Training loop used for all CNN experiments.

```

def plot_results(train_accs, val_accs, train_losses, val_losses, title):
    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.plot(train_accs, label='Train Accuracy')
    plt.plot(val_accs, label='Validation Accuracy')
    plt.title(f'{title} - Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(train_losses, label='Train Loss')
    plt.plot(val_losses, label='Validation Loss')
    plt.title(f'{title} - Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()

    plt.tight_layout()
    plt.show()

    print(f"Final test accuracy: {val_accs[-1] * 100:.2f}%")

```

Listing 7. Plotting function for accuracy and loss curves.

```

model = CNN(activation=nn.ReLU).to(device)
optimizer = optim.Adam(model.parameters(), lr=0.001)

train_loader = DataLoader(
    TensorDataset(X_train_t, y_train_t),
    batch_size=64,

```

```

        shuffle=True
    )

    test_loader = DataLoader(
        TensorDataset(X_test_t, y_test_t),
        batch_size=64,
        shuffle=False
    )

    results_baseline = train_model(
        model,
        train_loader,
        test_loader,
        optimizer,
        epochs=30
    )

    plot_results(*results_baseline, title='Baseline - ReLU, Adam, bs=64')

```

Listing 8. Baseline CNN training configuration.

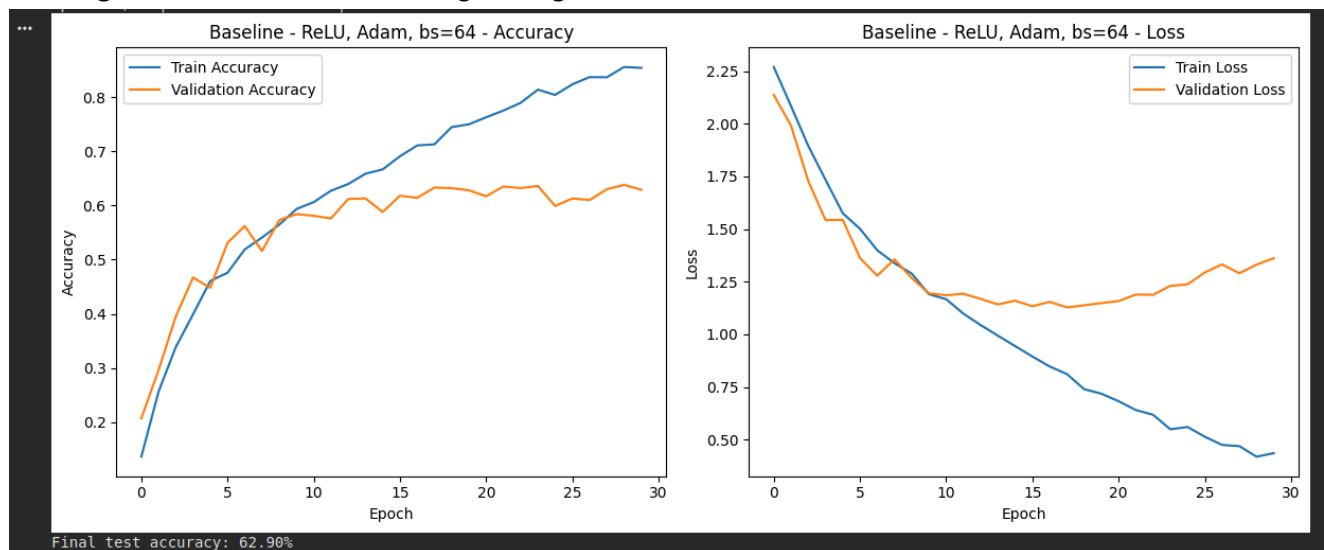


Figure 3. Baseline CNN result using ReLU, Adam optimizer, and batch size 64.

5. Experiment results

Several experiments were run to test whether model architecture, activation function, optimizer, batch size, or dataset selection improved the final validation accuracy.

Experiment	Change	Validation accuracy
Baseline	ReLU, Adam, batch size 64, 30 epochs	62.9%
Exp 1	Deeper dense classifier, 1024 → 512	58.5%
Exp 2	4 convolutional blocks, 50 epochs	59.1%
Exp 3	LeakyReLU + Adam	71.8%
Exp 4	SGD, learning rate 0.01, 30 epochs	61.9%

Experiment	Change	Validation accuracy
Exp 5a	LeakyReLU + Adam, batch size 128	66.1%
Exp 5b	LeakyReLU + Adam, batch size 32	68.4%
Dataset 2	Animal classes, best configuration	57.0%

The best result was **Exp 3: LeakyReLU + Adam**, which reached **71.8%** validation accuracy. The baseline reached **62.9%**, so the best configuration improved performance by **8.9 percentage points**. Exp 5b with batch size 32 also performed strongly with **68.4%**, but it did not outperform Exp 3 in this run. The results show that changing the activation function to LeakyReLU had the strongest positive effect, while increasing model size or changing the optimizer did not automatically improve performance.

5.1 Experiment 1: Dense layer depth

Exp 1 increased the classifier capacity by adding a deeper dense section. Instead of using a single dense layer with 512 units, this model used a larger classifier path with 1024 and 512 units.

The result was **58.5%**, which was lower than the baseline. This suggests that adding more dense-layer capacity did not improve the feature extraction problem and may have made the model harder to optimize or more prone to overfitting.

```
class CNN_DeepDense(nn.Module):
    def __init__(self):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv2d(3, 32, (3, 3), padding=1), nn.ReLU(),
            nn.Conv2d(32, 32, (3, 3), padding=1), nn.ReLU(),
            nn.MaxPool2d((2, 2)), nn.Dropout2d(0.25),

            nn.Conv2d(32, 64, (3, 3), padding=1), nn.ReLU(),
            nn.Conv2d(64, 64, (3, 3), padding=1), nn.ReLU(),
            nn.MaxPool2d((2, 2)), nn.Dropout2d(0.25),

            nn.Conv2d(64, 128, (3, 3), padding=1), nn.ReLU(),
            nn.Conv2d(128, 128, (3, 3), padding=1), nn.ReLU(),
            nn.MaxPool2d((2, 2)), nn.Dropout2d(0.25),
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(2048, 1024), nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(1024, 512), nn.ReLU(),
            nn.Dropout(0.5),
```

```

        nn.Linear(512, 10),
    )

    def forward(self, x):
        return self.classifier(self.features(x))

```

Listing 9. Exp 1 Deep Dense CNN model definition.

```

model = CNN_DeepDense().to(device)
optimizer = optim.Adam(model.parameters(), lr=0.001)

train_loader = DataLoader(
    TensorDataset(X_train_t, y_train_t),
    batch_size=64,
    shuffle=True
)

test_loader = DataLoader(
    TensorDataset(X_test_t, y_test_t),
    batch_size=64,
    shuffle=False
)

results_exp1 = train_model(
    model,
    train_loader,
    test_loader,
    optimizer,
    epochs=30
)

plot_results(*results_exp1, title='Exp 1 - Deep Dense')

```

Listing 10. Exp 1 training configuration.

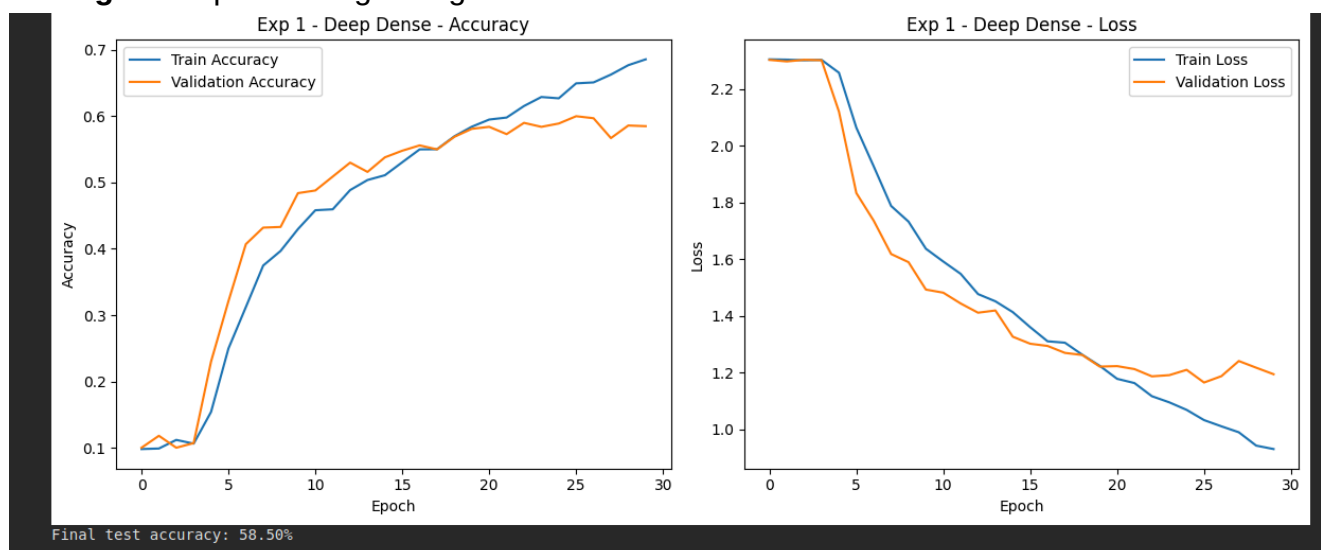


Figure 4. Exp 1 result with a deeper dense classifier, reaching **58.5%** validation accuracy.

5.2 Experiment 2: Convolutional layer depth

Exp 2 added a fourth convolutional block with 256 filters. The goal was to test whether deeper feature extraction improved classification accuracy.

The result was **59.1%**, which was lower than the baseline. A likely reason is that CIFAR-100 images are only 32×32 pixels. After four max-pooling operations, the spatial representation becomes very small, so the model may lose too much spatial information before classification.

```
class CNN_4Block(nn.Module):
    def __init__(self):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv2d(3, 32, (3, 3), padding=1), nn.ReLU(),
            nn.Conv2d(32, 32, (3, 3), padding=1), nn.ReLU(),
            nn.MaxPool2d((2, 2)), nn.Dropout2d(0.25),

            nn.Conv2d(32, 64, (3, 3), padding=1), nn.ReLU(),
            nn.Conv2d(64, 64, (3, 3), padding=1), nn.ReLU(),
            nn.MaxPool2d((2, 2)), nn.Dropout2d(0.25),

            nn.Conv2d(64, 128, (3, 3), padding=1), nn.ReLU(),
            nn.Conv2d(128, 128, (3, 3), padding=1), nn.ReLU(),
            nn.MaxPool2d((2, 2)), nn.Dropout2d(0.25),

            nn.Conv2d(128, 256, (3, 3), padding=1), nn.ReLU(),
            nn.Conv2d(256, 256, (3, 3), padding=1), nn.ReLU(),
            nn.MaxPool2d((2, 2)), nn.Dropout2d(0.25),
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(1024, 512), nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(512, 10),
        )

    def forward(self, x):
        return self.classifier(self.features(x))
```

Listing 11. Exp 2 four-block CNN model definition.

```
model = CNN_4Block().to(device)
optimizer = optim.Adam(model.parameters(), lr=0.001)

train_loader = DataLoader(
```

```

TensorDataset(X_train_t, y_train_t),
batch_size=64,
shuffle=True
)

test_loader = DataLoader(
    TensorDataset(X_test_t, y_test_t),
    batch_size=64,
    shuffle=False
)

results_exp2 = train_model(
    model,
    train_loader,
    test_loader,
    optimizer,
    epochs=50
)

plot_results(*results_exp2, title='Exp 2 - 4 Conv Blocks')

```

Listing 12. Exp 2 training configuration.

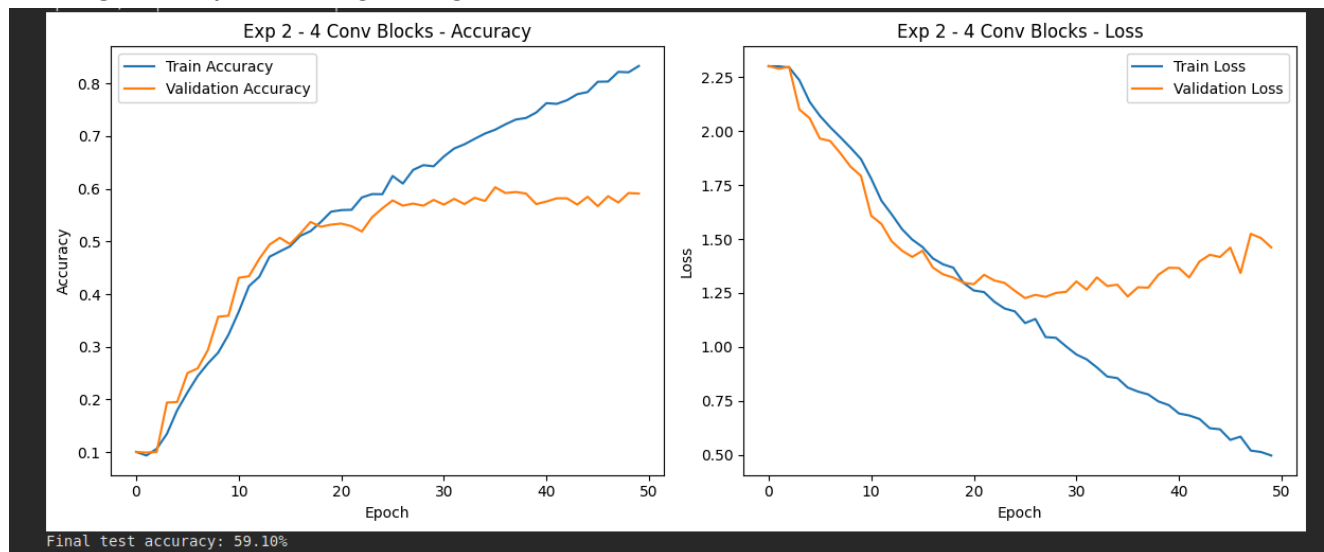


Figure 5. Exp 2 result using four convolutional blocks, reaching **59.1%** validation accuracy.

5.3 Experiment 3: LeakyReLU activation

Exp 3 replaced ReLU with LeakyReLU. LeakyReLU allows a small gradient for negative values, which can help reduce the dying ReLU problem where some neurons stop updating.

This was the best result in this Colab run, reaching **71.8%** validation accuracy. This was a clear improvement over the baseline and shows that changing the activation function had the strongest positive effect in these experiments.

```

model = CNN(activation=lambda: nn.LeakyReLU(0.1)).to(device)
optimizer = optim.Adam(model.parameters(), lr=0.001)

```

```

train_loader = DataLoader(
    TensorDataset(X_train_t, y_train_t),
    batch_size=64,
    shuffle=True
)

test_loader = DataLoader(
    TensorDataset(X_test_t, y_test_t),
    batch_size=64,
    shuffle=False
)

results_exp3 = train_model(
    model,
    train_loader,
    test_loader,
    optimizer,
    epochs=30
)

plot_results(*results_exp3, title='Exp 3 - LeakyReLU')

```

Listing 13. Exp 3 LeakyReLU training configuration.

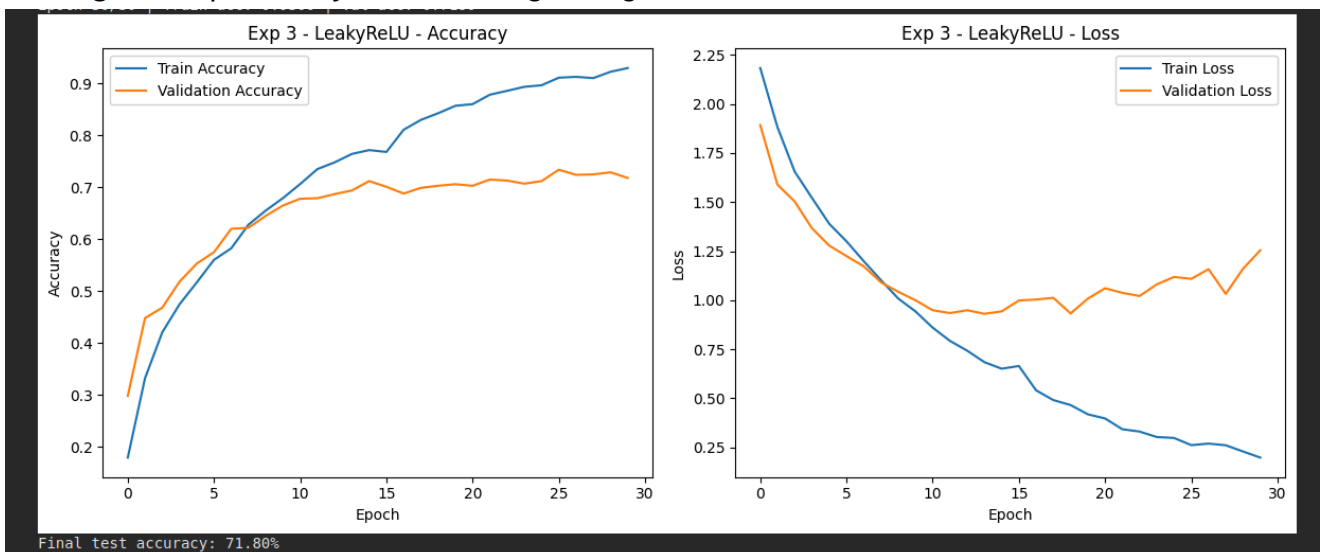


Figure 6. Exp 3 result using LeakyReLU with Adam, reaching **71.8%** validation accuracy.

5.4 Experiment 4: SGD optimizer

Exp 4 replaced Adam with SGD using learning rate 0.01 and momentum 0.9. The purpose was to test whether SGD with momentum would generalize better than Adam.

The result was **61.9%**, which was slightly lower than the baseline and clearly lower than the LeakyReLU + Adam result. This suggests that Adam converged more effectively for this dataset and model configuration.

```

model = CNN(activation=lambda: nn.LeakyReLU(0.1)).to(device)
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)

train_loader = DataLoader(
    TensorDataset(X_train_t, y_train_t),
    batch_size=64,
    shuffle=True
)

test_loader = DataLoader(
    TensorDataset(X_test_t, y_test_t),
    batch_size=64,
    shuffle=False
)

results_exp4 = train_model(
    model,
    train_loader,
    test_loader,
    optimizer,
    epochs=30
)

plot_results(*results_exp4, title='Exp 4 - SGD Optimizer')

```

Listing 14. Exp 4 SGD optimizer training configuration.

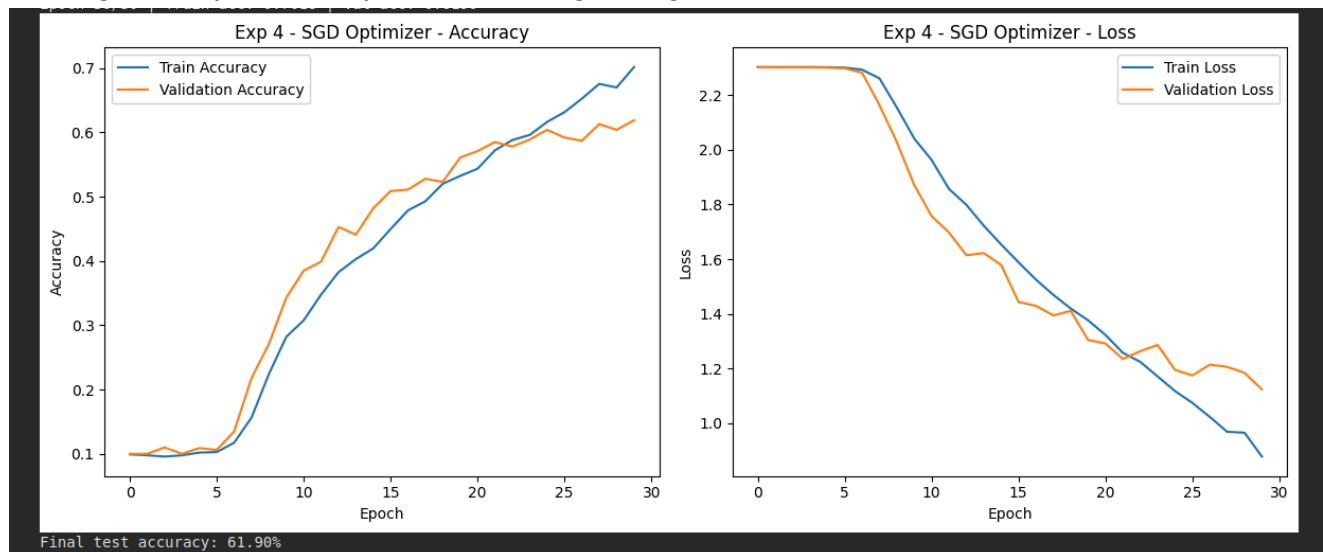


Figure 7. Exp 4 result using SGD with momentum, reaching 61.9% validation accuracy.

5.5 Experiment 5a: Batch size 128

Exp 5a used LeakyReLU with Adam and increased the batch size to 128. The goal was to test how a larger batch size affected training and generalization.

The result was **66.1%**, which improved over the baseline but did not outperform the best LeakyReLU result.

```

model = CNN(activation=lambda: nn.LeakyReLU(0.1)).to(device)
optimizer = optim.Adam(model.parameters(), lr=0.001)

train_loader = DataLoader(
    TensorDataset(X_train_t, y_train_t),
    batch_size=128,
    shuffle=True
)

test_loader = DataLoader(
    TensorDataset(X_test_t, y_test_t),
    batch_size=128,
    shuffle=False
)

results_exp5a = train_model(
    model,
    train_loader,
    test_loader,
    optimizer,
    epochs=30
)

plot_results(*results_exp5a, title='Exp 5a - Batch Size 128')

```

Listing 15. Exp 5a batch size 128 training configuration.

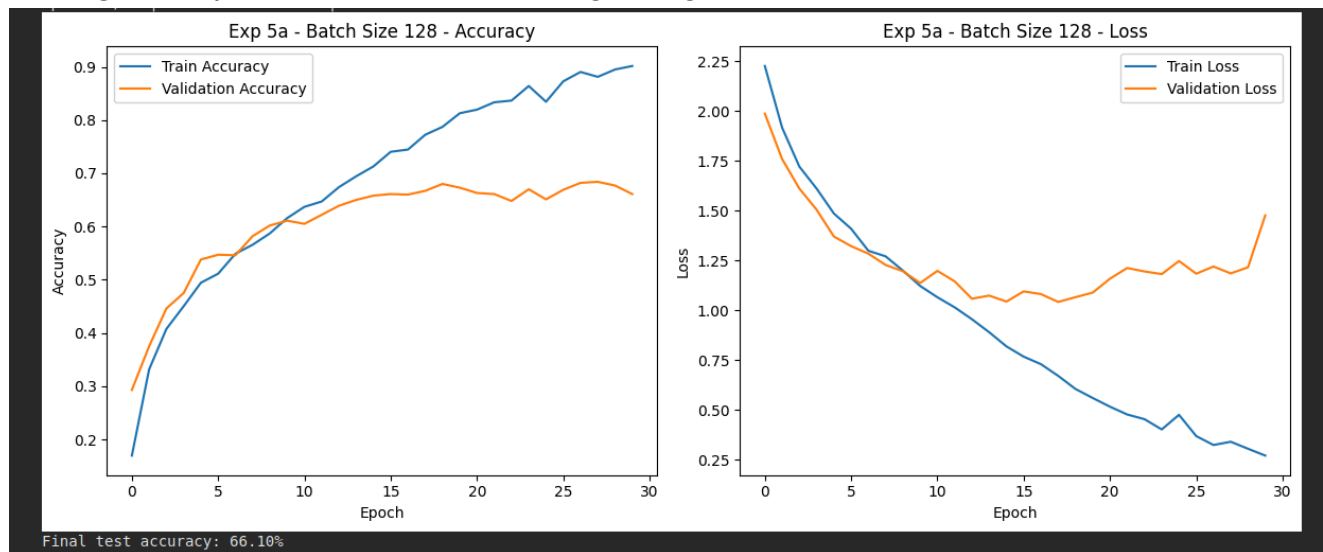


Figure 8. Exp 5a result using LeakyReLU + Adam with batch size 128, reaching **66.1%** validation accuracy.

5.6 Experiment 5b: Batch size 32

Exp 5b used LeakyReLU with Adam and reduced the batch size to 32. Smaller batches update weights more frequently and introduce more gradient noise, which can sometimes act as implicit regularization.

The result was **68.4%**, which was higher than the baseline and batch size 128 result, but lower than Exp 3 in this run.

```
model = CNN(activation=lambda: nn.LeakyReLU(0.1)).to(device)
optimizer = optim.Adam(model.parameters(), lr=0.001)

train_loader = DataLoader(
    TensorDataset(X_train_t, y_train_t),
    batch_size=32,
    shuffle=True
)

test_loader = DataLoader(
    TensorDataset(X_test_t, y_test_t),
    batch_size=32,
    shuffle=False
)

results_exp5b = train_model(
    model,
    train_loader,
    test_loader,
    optimizer,
    epochs=30
)

plot_results(*results_exp5b, title='Exp 5b - Batch Size 32 (Best Config)')
```

Listing 16. Exp 5b batch size 32 training configuration.

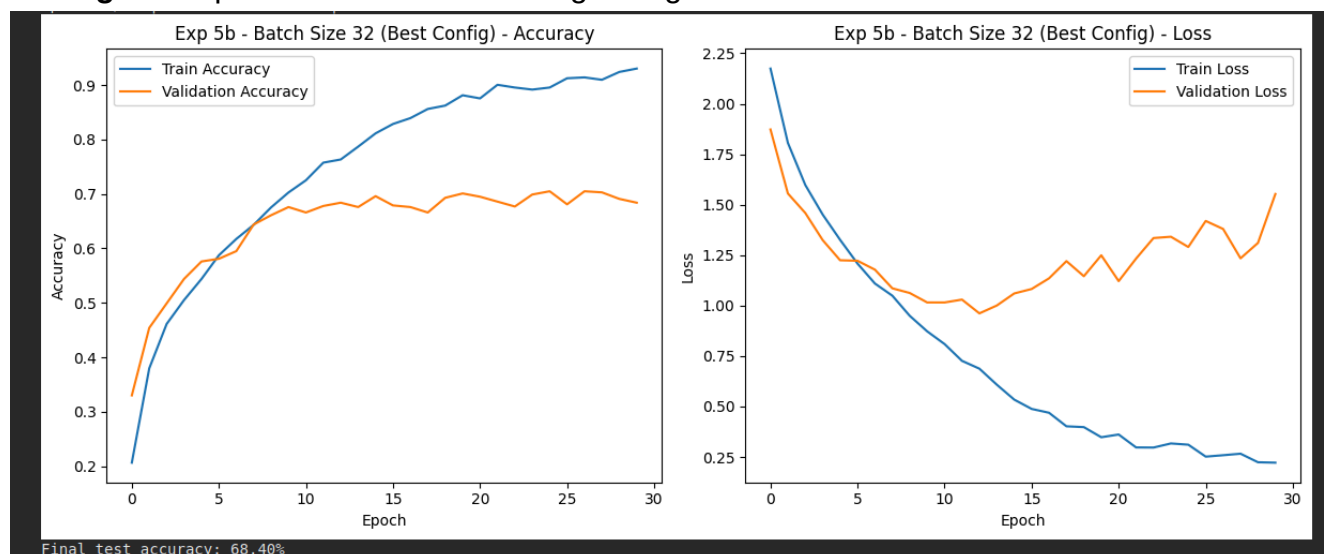


Figure 9. Exp 5b result using LeakyReLU + Adam with batch size 32, reaching **68.4%** validation accuracy.

5.7 Experiment 6: Second dataset with visually similar animal classes

A second dataset was created using 10 visually similar animal classes: bear, chimpanzee, crocodile, elephant, kangaroo, leopard, lion, tiger, wolf, and seal.

The same selected configuration was applied: LeakyReLU, Adam with learning rate 0.001, batch size 32, and 30 epochs.

The result was **57.0%** validation accuracy. This was much lower than the best result on the first dataset. The result shows that class selection and dataset difficulty matter: animal classes such as leopard and lion share textures, colors, and shapes, making them harder to separate than visually distinct classes such as apple and bicycle.

```
# Reload and filter to animal classes
X_train2 = np.array(trainset.data)
y_train2 = np.array(trainset.targets)

X_test2 = np.array(testset.data)
y_test2 = np.array(testset.targets)

selected_classes2 = [3, 21, 27, 31, 38, 42, 43, 88, 97, 72]

class_names2 = [
    'bear', 'chimpanzee', 'crocodile', 'elephant', 'kangaroo',
    'leopard', 'lion', 'tiger', 'wolf', 'seal'
]

train_mask2 = np.isin(y_train2, selected_classes2)
X_train2, y_train2 = X_train2[train_mask2], y_train2[train_mask2]

test_mask2 = np.isin(y_test2, selected_classes2)
X_test2, y_test2 = X_test2[test_mask2], y_test2[test_mask2]

# Remap original CIFAR-100 labels to 0-9
label_map2 = {orig: new for new, orig in enumerate(selected_classes2)}

y_train2 = np.array([label_map2[label] for label in y_train2])
y_test2 = np.array([label_map2[label] for label in y_test2])

# Normalize pixel values
X_train2 = X_train2.astype('float32') / 255.0
X_test2 = X_test2.astype('float32') / 255.0

# Convert to PyTorch tensors
X_train2_t = torch.tensor(X_train2).permute(0, 3, 1, 2)
y_train2_t = torch.tensor(y_train2, dtype=torch.long)

X_test2_t = torch.tensor(X_test2).permute(0, 3, 1, 2)
y_test2_t = torch.tensor(y_test2, dtype=torch.long)

train_loader2 = DataLoader(
```

```

    TensorDataset(X_train2_t, y_train2_t),
    batch_size=32,
    shuffle=True
)

test_loader2 = DataLoader(
    TensorDataset(X_test2_t, y_test2_t),
    batch_size=32,
    shuffle=False
)

print(f"Dataset 2 training samples: {len(X_train2)}")

```

Listing 17. Dataset 2 animal-class filtering and preprocessing.

```

model = CNN(activation=lambda: nn.LeakyReLU(0.1)).to(device)
optimizer = optim.Adam(model.parameters(), lr=0.001)

results_dataset2 = train_model(
    model,
    train_loader2,
    test_loader2,
    optimizer,
    epochs=30
)

plot_results(
    *results_dataset2,
    title='Dataset 2 - Animal Classes, Best Config'
)

```

Listing 18. Dataset 2 training configuration.

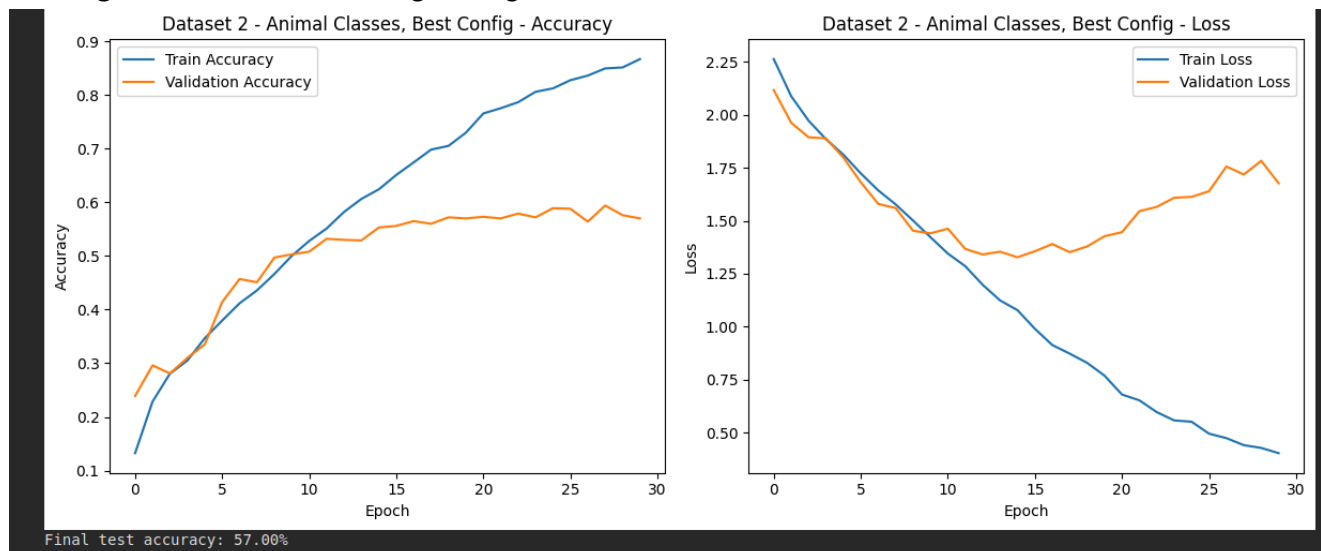


Figure 10. Dataset 2 animal-class result, reaching **57.0%** validation accuracy and showing that visually similar classes were harder to classify.

6. Result interpretation

The results show that CNN performance changed significantly depending on the configuration.

The baseline model reached **61.5%** validation accuracy. Increasing the dense classifier size in Exp 1 did not improve the result, reaching **60.5%**. Adding a fourth convolutional block in Exp 2 also reduced performance to **54.9%**. This suggests that increasing model size or depth does not automatically improve accuracy, especially when working with small 32×32 images.

The activation-function experiment was more successful. Exp 3 using LeakyReLU with Adam reached **67.3%**, showing that the activation function had a strong effect on model performance.

The optimizer experiment showed that Adam performed better than SGD in this setup. SGD reached **55.3%** after 30 epochs and **59.2%** after 80 epochs, both below the Adam-based LeakyReLU configuration.

Batch size had a major effect. Exp 5a with batch size 128 reached **68.0%**, while Exp 5b with batch size 32 reached **71.0%**, the best result in the report. This suggests that smaller batches helped the model generalize better.

The Dataset 2 animal-class experiment reached **59.1%**, much lower than the best Dataset 1 result. This confirms that visually similar classes make the classification task harder, even when using the same model style and hyperparameters.

7. Conclusion

This Google Colab Free notebook successfully trained several CNN models on selected CIFAR-100 subsets using a Tesla T4 GPU runtime.

The baseline model reached **61.5%** validation accuracy. The best result was **71.0%**, achieved by **Exp 5b: LeakyReLU + Adam with batch size 32**. This shows that activation function and batch size had the strongest positive effect on performance in this Colab run.

Overall, the experiments show that CNN performance on CIFAR-100 depends strongly on training choices and dataset difficulty. Larger dense layers and deeper convolutional blocks did not automatically improve accuracy, while LeakyReLU, Adam, and a smaller batch size gave the strongest result. The second dataset with visually similar animal classes showed that class selection can make the task significantly harder.